



Derridj mellyna

Loic Weber

Préparation: échantillonnage d'une séquence

On considère un signal sinusoïdal d'amplitude 1 et de fréquence 10Hz échantillonné à une fréquence $f_e=1\text{kHz}$.

```
fe = 1000      # fréquence d'échantillonnage
f0 = 10        # fréquence du signal
Dobs = 0.5     # durée d'observation

t = np.arange(0, Dobs, 1/fe)
s = np.sin(2 * np.pi * f0 * t)
```

On réalise un signal s_2 échantillonné cette fois-ci à une fréquence $f_e=100\text{Hz}$

```
fe2 = 100
t2 = np.arange(0, Dobs, 1/fe2)
s2 = np.sin(2 * np.pi * f0 * t2)
```

Enfin, on crée un signal s_3 qui échantille tous les 10 le signal s .

```
Nsous = np.arange(0, len(s), 10)

s3 = s[Nsous]
t3 = t[Nsous]
```

On obtient les signaux suivants :

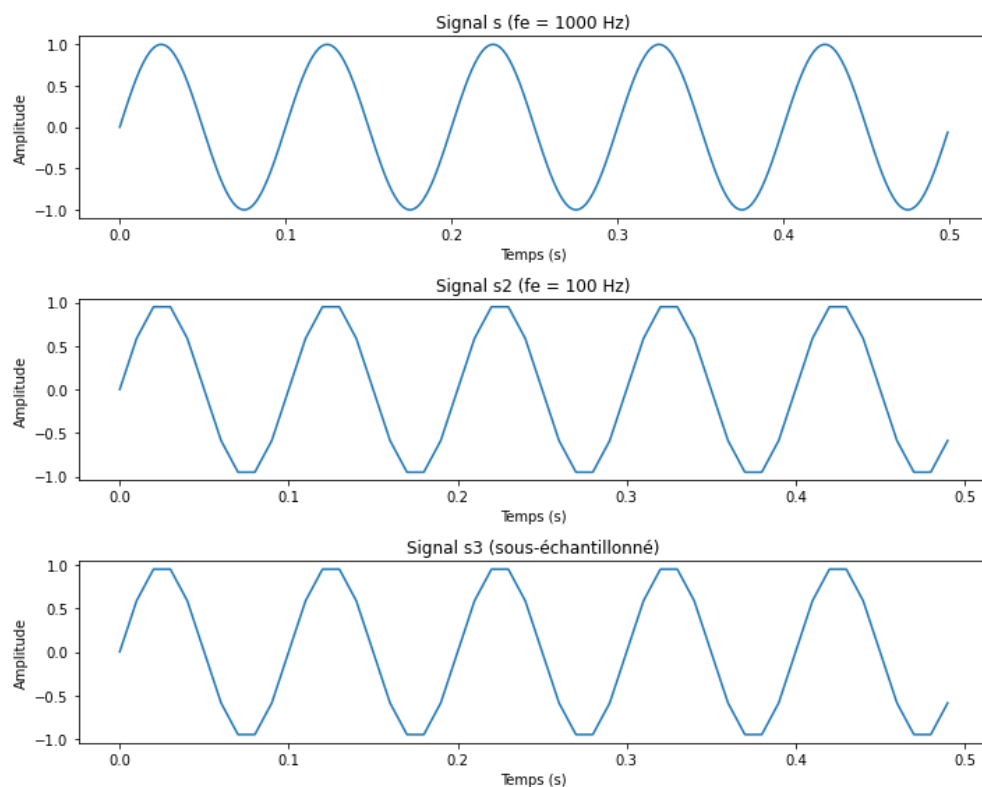


Fig1.1 échantillonnage des signaux

Remarque : le critère de Shannon est bien vérifié pour tous les signaux $f_e \geq 2f_{\max} = 20\text{Hz}$ donc le signal est correctement échantillonné, pas de repliement de spectre.

Analyse : les signaux s_1 et s_2 sont différents (cohérent car la fréquence d'échantillonnage n'est pas la même). Cependant, les signaux s_2 et s_3 sont les mêmes car on échantillonne toutes les 10 fois un signal de fréquence d'échantillonnage $f_e = 1000\text{Hz}$. Finalement, la fréquence d'échantillonnage de ce signal est de $f_e = 100\text{Hz}$ (la même que celle de s_2) d'où le fait qu'ils soient identiques.

```
s==s2  
Out[56]: False
```

```
In [58]: s2==s3  
Out[58]:  
array([ True,  True,  True,  True,  True,  True,  True,  True,  True,  
        True,  True,  True,  True,  True,  True,  True,  True,  True,  
        True,  True,  True,  True,  True,  True,  True,  True,  True,  
        True,  True,  True,  True,  True,  True,  True,  True,  True,  
        True,  True,  True,  True,  True])
```

IV. 2.2 Représentation d'une séquence sinusoïdale

A partir du document fournit sur Moodle (sequence.npz) on cherche à reconstituer le signal en temporel puis en fréquentiel.

On relève donc la fréquence d'échantillonnage f_e , Nobs et un vecteur contenant les amplitudes du signal (échantillonnée tous les $1/f_e$) puis on réalise sa TFD.

```
#Données
donnee= load ("C:/Users/33658/Documents/sequence.npz")
|
se_nobs=donnee['se_nobs']
se_fe=donnee['se_fe']
se_amp= donnee['se_amp']

Te = 1 / se_fe
se_nobs = len(se_amp)

Dobs = se_nobs * Te

#Transformée de fourier
t = linspace(0,se_nobs/se_fe, len(se_amp))
Se = fft.fft(se_amp) / se_fe
freqs = fft.fftfreq(se_nobs, Te)
```

Fig1.1 Code d'affichage du signal $s(t)$ et de sa TFD

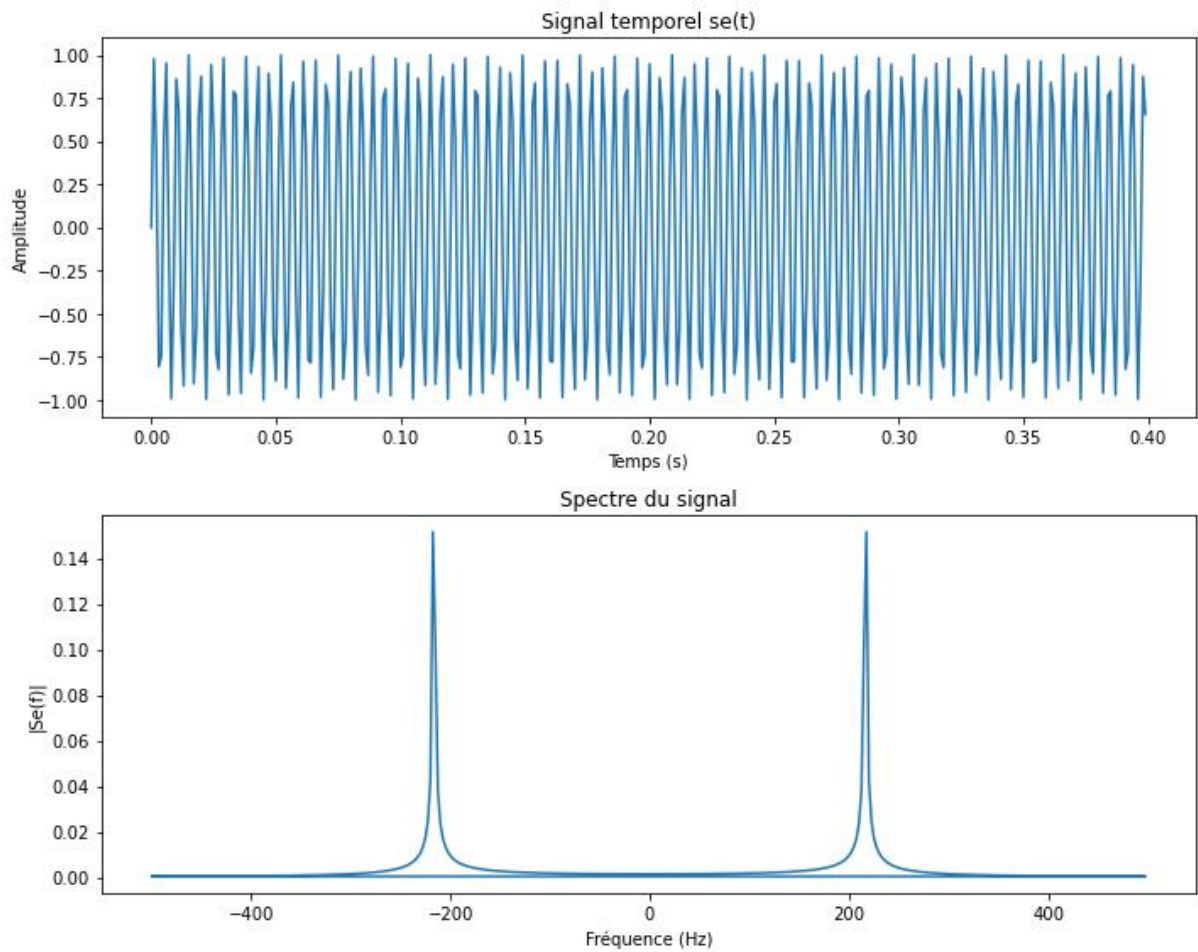


Fig 2.2 : Signal $s(t)$ et sa transformée de Fourier

Commentaire :

Le signal recomposé se présente comme une sinusoïdale. On sait que la transformée de fourrier d'un signal sinusoïdal est une somme deux Dirac :

$$\frac{1}{2.j} \cdot (e^{j \cdot \varphi_0} \cdot \delta(f - f_0) - e^{-j \cdot \varphi_0} \cdot \delta(f + f_0))$$

En valeur absolue, on est donc censé obtenir deux pics en + et - f_0 symétrique (Cohérent avec les résultats obtenus)

IV.2.3 Influence du nombre d'échantillon Nobs

On extrait une séquence *sm* du signal *se*. (Puisque non précisé, on prend une fenêtre de 0 à 100 soit 101 valeurs). On réalise ensuite la transformée de fourier de cet extrait.

```
#IV.2.3

#donnée
sm =se_amp[0:100]
sm_nobs = len(sm)
t2 = linspace(0,sm_nobs/se_fe, sm_nobs)

#Transformée de Fourier
Sm = fft.fft(sm) /se_fe # TFD= 1/fe TFR#
Sm_shifted = fft.fftshift(Sm)
freqs2 = fft.fftshift(fft.fftfreq(sm_nobs, Te))
```

*Fig 3.1 Transformée de fourier d'un fenêtrage de **se***

On obtient les résultats suivants :

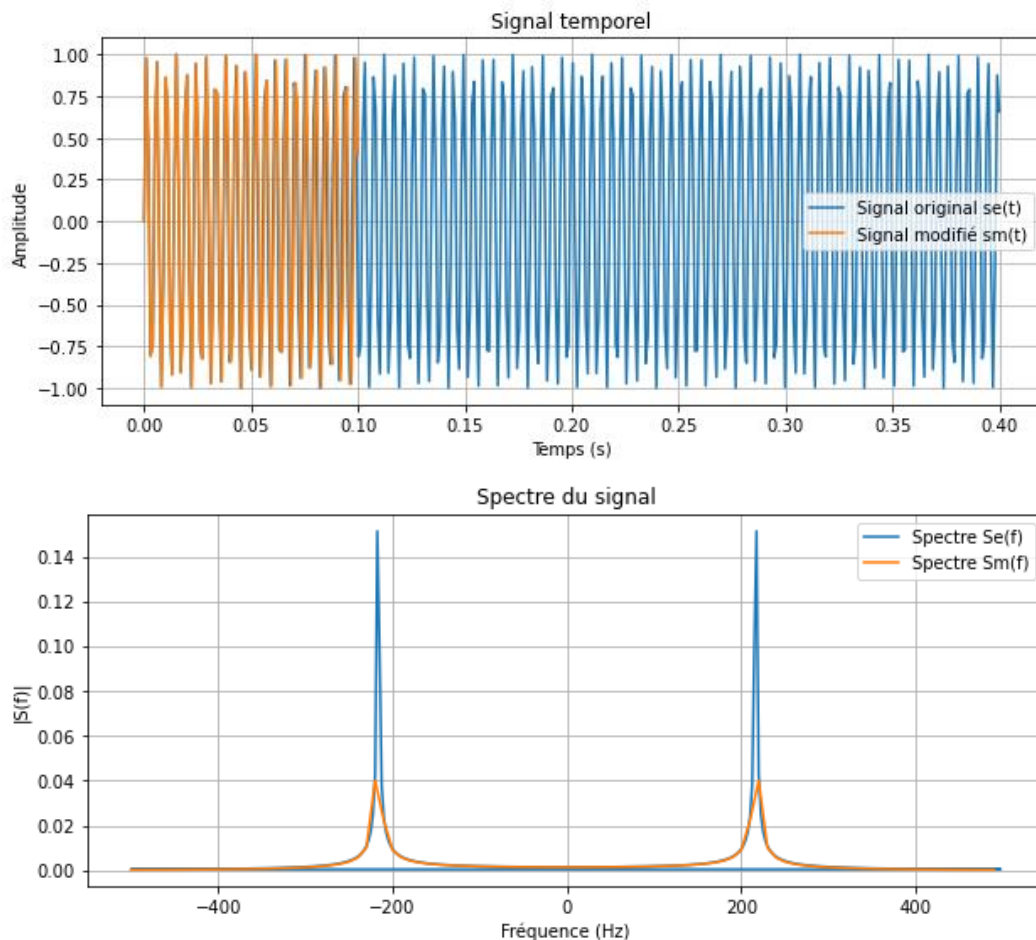


Fig3.2 affichages de sm et de sa Tf

Commentaire :

La transformée de fourier du signal extrait et de celui d'origine ont un **pic au même endroit**. Cohérent car sm et se ont la même "forme" : deux sinusoïdale de même fréquence.

En revanche, les **amplitudes diffèrent** car l'amplitude de la TFD est une somme (en prenant en valeur absolue, ce sont uniquement des termes positif). Si on prend un échantillon, la somme est plus petite. D'où le fait que l'amplitude de la TFD de l'échantillon soit plus petite.

IV.2.4 Influence de l'ordre de Nfft de la FFT

On modifie la longueur du signal dont on calcule la transformée de fourier pour créer f0

```

Nfft = 8*se_nobs

#Transformée de fourier |
So = fft.fft(se_amp, n=Nfft) / se_fe
freqs3 = fft.fftfreq(Nfft, Te)
figure()

plot(freqs3, abs(So), label="Spectre So(f)")
xlim(200, 230) #pour zoomer sur le pic
xlabel("Fréquence (Hz)")
ylabel("|Se(f)|")
title("Spectre du signal")
grid()
legend()

show()

```

Fig 4.1 Transformée de fourier de f_0

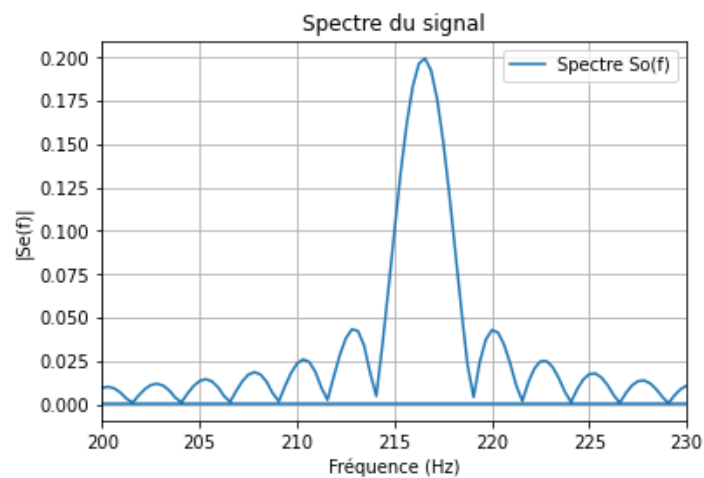


Fig 4.2 Affichage (avec zoom) du pic de la TF de S_o

Commentaire : sera fait dans la partie juste en-dessous.

V.2.3 Influence du zero-padding

On modifie encore une fois Nfft et on concatène le signal de base avec le nouveau Nfft .

```

Nfft2=7*se_nobs

conc=zeros(Nfft2)
szp=concatenate((se_amp,conc))

t4 = arange(Nfft) * Te

Sz = fft.fft(se_amp, n=Nfft) / se_fe
freqs4 = fft.fftfreq(Nfft, Te)

```

Fig 5.1 Code de la fonction Se

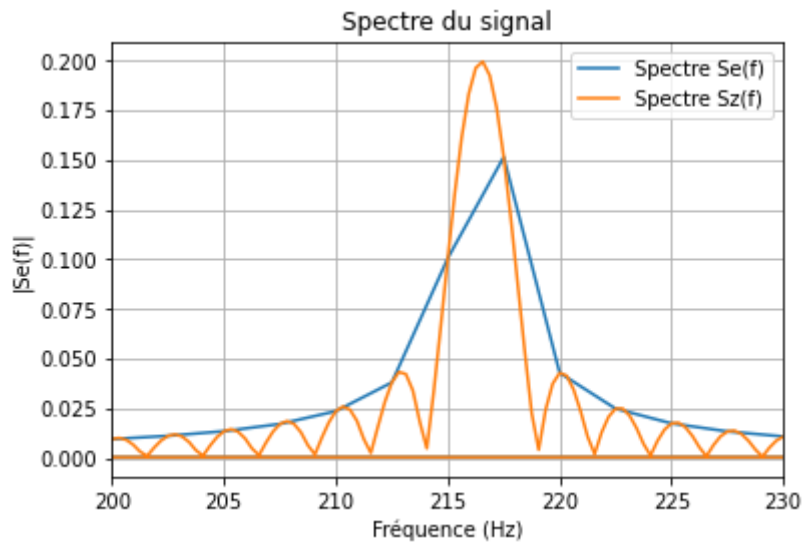


Fig 5.2 Affichage de la TFD

Commentaire : Lorsque on tronque le signal d'origine, cela équivaut à multiplier le signal d'origine en temporel par une porte. Or, la transformée de fourier d'un produit est le produit de convolution des transformée de fourier (d'une porte et d'un sinus en l'occurrence) Or, la TF d'une porte est un sinus cardinal (et celle d'un sinus une somme de Dirac), d'où la présence d'un sinus cardinal.